Week 5: Securing Web Application Data

Learning Objectives

By the end of this lecture, the student should be able to:

1. *Distinguish encoding, encryption, and hashing, and identify the appropriate use case for each technique.*
2. *Explain the difference between protecting data in transit and protecting data at rest.*
3. *Describe how symmetric and asymmetric encryption work together inside the TLS handshake introduced in Week 1.*
4. *Apply data classification levels to determine which protections a given category of data requires.*
5. *Explain the core principles of key management, including secure storage, rotation, and the danger of hard coded keys.*
6. *Describe tokenization and data minimization as practical strategies for reducing exposure.*
7. *Connect data protection failures to OWASP A02:2021, Cryptographic Failures.*

Continuity from Week Four

Week Four examined the architecture of a web application as a chain of components, each guarded by a trust boundary. This lecture narrows the focus to one specific asset that moves through every layer of that chain: the data itself. The question addressed here is not who may access a system, but how the information inside that system is protected once an access decision has already been made.

Data Classification as a Starting Point

Before any protection can be applied, the data itself must be classified. A common scheme divides information into four levels: public data, meant for anyone; internal data, meant only for staff or authenticated users; confidential data, such as financial records or business plans; and restricted data, which includes personally identifiable information, health records, and payment details. The classification of a given field determines the strength of protection it requires. Treating a public marketing page and a customer's national identification number with the same level of care wastes effort in one direction and creates unnecessary risk in the other. Data classification schemes vary between organizations; the four-level model below is a simplified example used to illustrate protection requirements.

Level	Example	Required protection
Public	Marketing pages, published pricing	Integrity checks only; confidentiality is not a concern.
Internal	Internal documentation, non-public reports	Access control and authentication.
Confidential	Financial records, business strategy	Access control plus encryption at rest.
Restricted	National ID numbers, health records, payment card data	Encryption at rest and in transit, strict access logging, and often a regulatory obligation.

Protecting Data in Transit and at Rest

Data protection is generally described along two dimensions. Data in transit is information moving across a network: between the browser and the server, or between internal services. Weeks 1 and 2 already introduced the primary defense for this dimension, TLS, which wraps the HTTP conversation in encryption so that a party observing the network traffic cannot read or alter it. Data at rest is information sitting in storage: a database, a file system, a backup, or a cache. This dimension is easy to overlook, since a system can enforce HTTPS everywhere and still store an entire customer table as plain text on disk. Both dimensions require independent protection; strength in one does not compensate for weakness in the other.

Distinguishing Encoding, Encryption, and Hashing

These three terms are frequently confused, and the distinction carries direct security consequences. Encoding transforms data into a different format for compatibility reasons, such as Base64. It uses no secret key and is trivially reversible by anyone, so it provides no confidentiality at all. Encryption transforms data using a secret key, so that only a party holding the correct key can reverse the transformation and recover the original value. Hashing, introduced in Week 3 for password storage, is a one-way transformation with no key and no reversal; it is used to verify integrity or to compare values without ever storing the original. A cryptographic hash by itself does not provide authenticity.

Technique	Reversible	Requires a key	Typical use
Encoding	Yes, by anyone	No	Data format compatibility, such as Base64 or URL encoding.
Encryption	Yes, only with the key	Yes	Protecting the confidentiality of data in transit or at rest.
Hashing	No	No	Password storage (Week 3) and integrity verification.

Symmetric and Asymmetric Encryption in Practice

Encryption itself divides into two families. Symmetric encryption uses a single shared key for both encrypting and decrypting; it is fast and well suited to large volumes of data, with AES as the standard example. Asymmetric encryption uses a mathematically linked key pair, a public key that may be shared freely and a private key that must remain secret; data encrypted with the public key can be decrypted only with the matching private key. Asymmetric operations are computationally expensive compared to symmetric ones, which is why TLS, introduced in Week 1, uses both in sequence rather than choosing one.

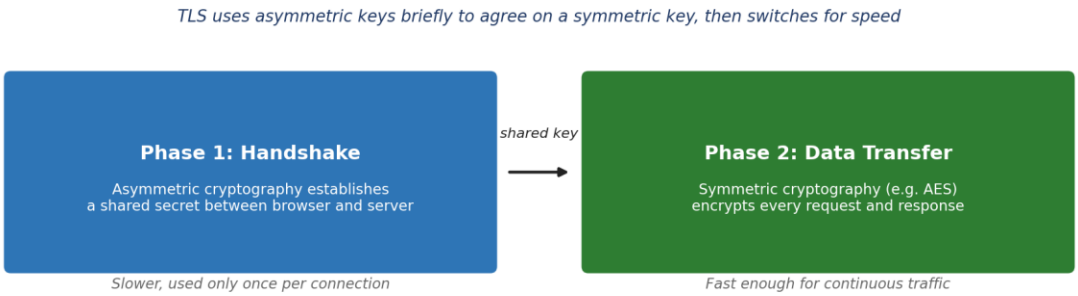


Figure 1. The handshake phase relies on asymmetric cryptography only long enough to agree on a shared session key. Every following byte of the conversation is then protected with fast symmetric encryption.

Modern TLS uses asymmetric cryptography primarily for authentication and digital signatures, while ephemeral Diffie–Hellman key exchange is used to establish shared session keys. After the handshake, symmetric cryptography protects the application data because it is much faster.

Principles of Key Management

An encryption scheme is only as strong as the protection around its key. Three principles recur across nearly every real-world failure in this area. First, a key must never be stored alongside the data it protects, since an attacker who steals the data would then also hold the means to read it; keys belong in a dedicated key management service or hardware security module, consistent with the secrets manager introduced in Week 4. Second, keys must be rotated on a schedule, so that a key exposed at some point in the past does not remain valid indefinitely. Third, a key must never be hard coded into source code or configuration files checked into version control, since doing so effectively publishes the key to anyone with repository access, present or future.

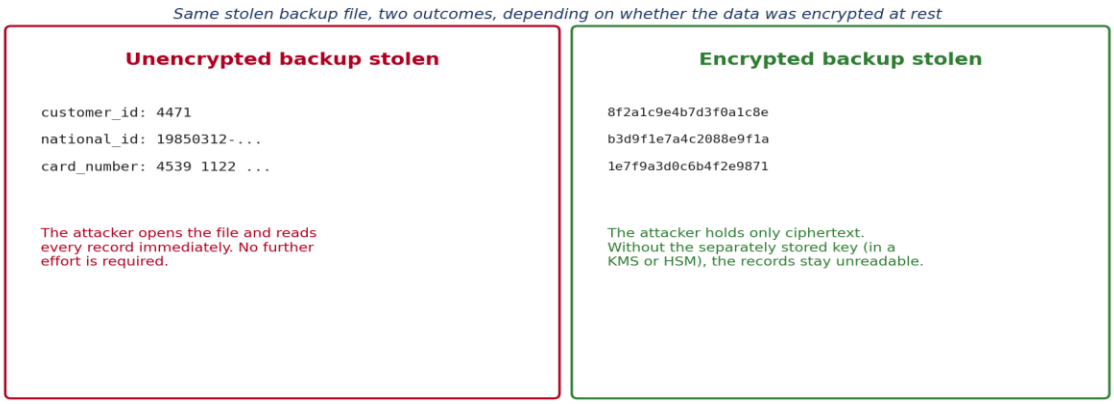


Figure 2. The same stolen backup produces two very different outcomes, depending entirely on whether encryption at rest was applied and the key was stored separately.

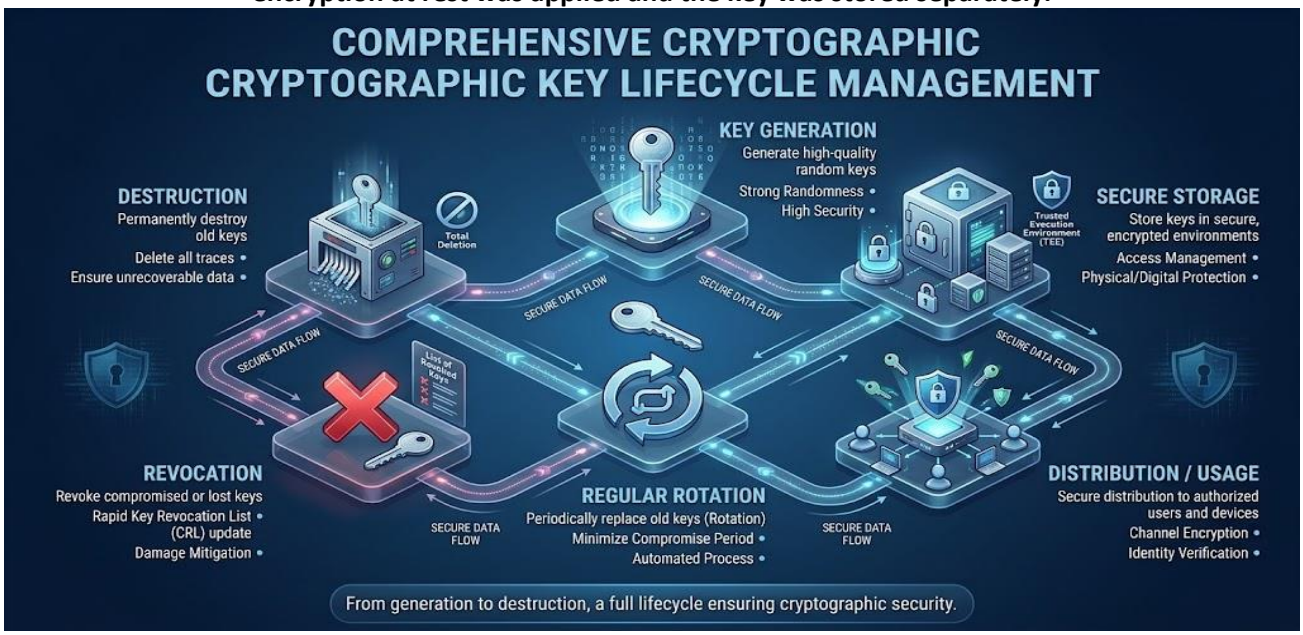


Figure 3. The key lifecycle.

Tokenization and Data Minimization

Two further strategies reduce exposure without relying purely on cryptography. Tokenization replaces a sensitive value, most commonly a payment card number, with a randomly generated token that carries no mathematical relationship to the original. The mapping between token and real value is kept in a separate, tightly controlled vault. If the application database is later breached, the attacker recovers only meaningless tokens rather than usable card numbers. Tokenization is widely used in payment systems to reduce the exposure of sensitive card data and can help organizations meet applicable compliance requirements. Data minimization addresses the same problem from a different angle: collect only the data a feature genuinely requires, and delete it once its purpose has been fulfilled. Data that was never collected, or that has already been deleted, cannot be exposed in a future breach.

Cryptographic Failures in the OWASP Top Ten

This week's material corresponds to OWASP A02:2021, Cryptographic Failures. The category covers transmitting sensitive data in plain text rather than over TLS, applying outdated or weak algorithms such as MD5 or DES, leaving sensitive data unencrypted at rest, storing or generating keys improperly, and relying on predictable random number generation where a cryptographically secure source was required. Nearly every item in this list is a direct violation of a principle introduced earlier in this lecture.

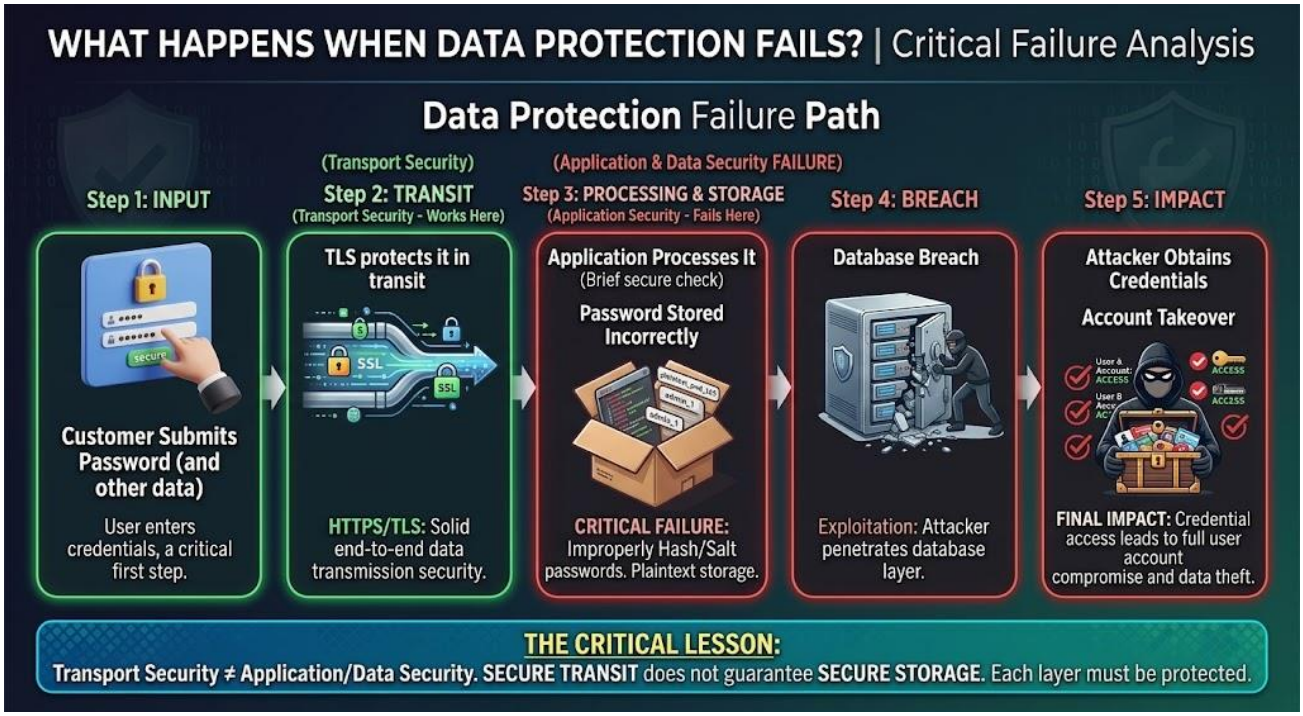


Figure 4. Protection failure analysis.